

Дополнительные главы дискретной математики:

Алгоритмы и структуры данных

Лекция 5. Амортизационный анализ и косые деревья

Ключарёв Пётр Георгиевич

pgkl@yandex.ru

twitter.com/klyucharev

Кафедра ИУ8 «Информационная безопасность»
МГТУ им. Н.Э. Баумана

2013 г.

Амортизационный анализ

Амортизационный анализ

Амортизационный анализ применяется для оценки времени выполнения нескольких операций с какой-либо структурой данных (или, что равносильно, среднего времени выполнения одной операции).

Учетная стоимость

При амортизационном анализе каждой операции присваивается некоторая *учётная стоимость* (amortized cost), которая может отличаться от реальной сложности операции.

Для любой последовательности операций фактическая суммарная длительность всех операций не должна превосходить суммы их учётных стоимостей. Для одной и той же структуры данных и одних и тех же алгоритмов можно корректно назначить учётные стоимости различными способами.

Метод группировки

Общие сведения

Метод группировки (aggregate method) состоит в следующем: для каждого n оценивается время $T(n)$, требуемое на выполнение n операций (в худшем случае). Время в расчёте на одну операцию, то есть отношение $T(n)/n$, объявляется учётной стоимостью одной операции. При этом учётные стоимости всех операций оказываются одинаковыми.

Стек

Рассмотрим стек S с тремя операциями:

- $push(S, x)$
- $pop(S)$
- $multipop(S, k)$ — k раз выполняет $pop(S)$.

Операции $push$ и pop требуют по единице времени.

Метод группировки: пример

Анализ стека методом группировки

Оценим суммарную стоимость n операций, применённых к изначально пустому стеку. Самая сложная операция *multipop* стоит не более n , поскольку за n операций в стеке не наберётся более n элементов). Следовательно, суммарная стоимость n операций есть $O(n^2)$. Но это очень грубая оценка.

Заметим, что поскольку в начале стек был пуст, общее число реально выполняемых операций pop, включая вызываемые из процедуры multipop не превосходит общего числа операций push, которое не превосходит n . Следовательно, стоимость последовательности из n операций push, Pop и Multipop есть $O(n)$. Поэтому, учетная стоимость каждой из операций есть $O(n)/n = O(1)$.

Метод потенциалов

Пусть над структурой данных предстоит произвести последовательность из n операций. D_i — состояние структуры данных после i -й операции (D_0 — исходное состояние).

Потенциал (потенциальная функция) — это функция Φ отображающая множество состояний структуры данных во множество $\mathbb{R}$.

Пусть c_i — реальная стоимость i -й операции. *Учетной стоимостью* i -й операции объявим число

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}), \quad (1)$$

Тогда суммарная учетная стоимость всех операций равна

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0). \quad (2)$$

Если $\Phi(D_n) \geq \Phi(D_0)$, то суммарная учетная стоимость даст верхнюю оценку реальной стоимости последовательности

n операций

Метод потенциалов: пример

В качестве потенциальной функции Φ возьмем количество элементов в стеке. Поскольку мы начинаем с пустого стека, имеем

$$\Phi(D_i) \geq 0 = \Phi(D_0).$$

Поэтому, сумма учётных стоимостей оценивает сверху реальную стоимость последовательности операций.

Найдем теперь учётные стоимости индивидуальных операций. Так как операция `push` имеет реальную стоимость 1 и увеличивает потенциал на 1, её учётная стоимость равна $1 + 1 = 2$; операция `Multipop(S, m)` имеет стоимость m , но уменьшает потенциал на m , поэтому её учётная стоимость равна $m - m = 0$. Учетная стоимость операции `Pop` равна нулю. Итак, учётная стоимость каждой операции не превосходит 2, поэтому стоимость последовательности из n операций, есть $O(n)$.

Косое дерево

Косое дерево

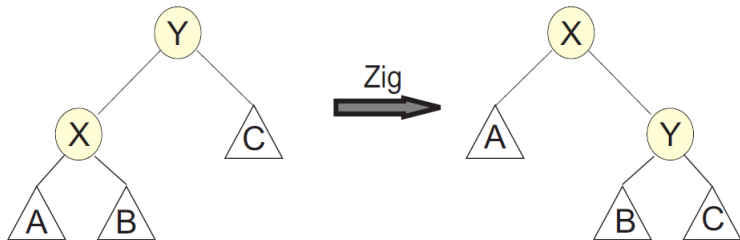
Косое дерево (splay tree) — это двоичное дерево поиска, основные операции над которым модифицируют структуру дерева таким образом, чтобы элемент, к которому осуществлялся доступ последний раз оказался корнем дерева.

Перекас дерева

Перекас — операция изменения структуры дерева, так что определенная вершина становится корнем дерева.

Zig

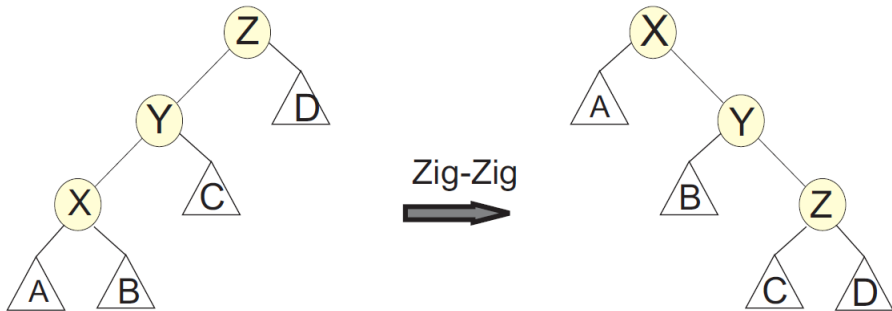
Вершина x — ребенок корня дерева.



ZigZig

Вершина $x.p$ не является корнем дерева.

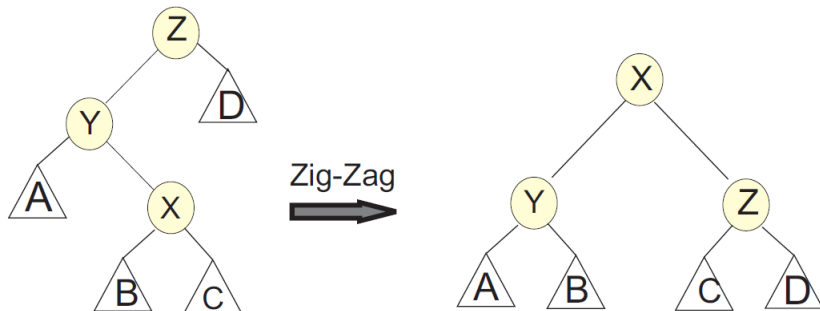
Вершины x и $x.p$ — оба левые или оба правые дети.



ZigZag

Вершина $x.p$ не является корнем дерева.

Вершина x — левый ребенок, а вершина $x.p$ — правый ребенок, или наоборот.



Перекося дерева (*Splay*)

Алгоритм 1 Перекося дерева (*Splay*)

```
1: procedure SPLAY( $T, x$ )
2:   while  $x \neq T.root$  do
3:     if  $x.p$  является корнем дерева  $T$  then
4:        $Zig(x)$ 
5:     else
6:       if  $x$  и  $x.p$  — оба левые дети then
7:          $ZigZigL(x)$ 
8:       else if  $x$  и  $x.p$  — оба правые дети then
9:          $ZigZigR(x)$ 
10:      else if  $x$  — левый ребенок, а  $x.p$  — правый ребенок then
11:         $ZigZagL(x)$ 
12:      else if  $x$  — правый ребенок, а  $x.p$  — левый ребенок then
13:         $ZigZagR(x)$ 
```

Поиск и добавление вершины

Поиск

- 1 Выполнить поиск как обычно.
- 2 Если вершина найдена, выполнить для нее процедуру *Splay*.
Если не найдена — выполнить *Splay* для последней вершины на пути поиска.

Добавление вершины

- 1 Выполнить добавление вершины как обычно.
- 2 Выполнить для добавленной вершины процедуру *Splay*.

Удаление вершины

Удаление вершины x из дерева T

- 1 Вызывается $Splay(T, x)$.
- 2 Удаляется корень из дерева. Остаются два поддерева: левое L и правое R .
- 3 В L ищется вершина z с максимальным ключом.
- 4 Вызывается $Splay(T, z)$.
- 5 Теперь z — корень дерева, но у него нет правого поддерева. В качестве правого поддерева пристыковывается дерево R .

Анализ *Splay*

Обозначения

$w(x)$ — вес вершины x .

$s(x)$ — сумма весов всех вершин в поддереве с корнем x .

$r(x) = \log s(x)$ — ранг вершины x .

Будем обозначать w', s', r' — соответствующие вершины после выполнения *Splay*.

Метод

Воспользуемся методом потенциалов.

$$\sum_{i=1}^n c_i = \sum_{i=1}^n \hat{c}_i + \Phi(D_0) - \Phi(D_n). \quad (3)$$

Лемма о доступе

Лемма (о доступе)

Учетное время выполнения процедуры $Splay(T, x)$ для дерева T с корнем t не превышает $3(r(t) - r(x)) + 1 = O(\log(s(t)/s(x)))$.

Доказательство.

Будем считать, что одно вращение происходит за единицу времени. Случай Zig. Одно вращение. Изменяются ранги только двух вершин: x и y .

Учетное время:

$1 + r'(x) + r'(y) - r(x) - r(y) \leq 1 + r'(x) - r(x) \leq 1 + 3(r'(x) - r(x))$,
поскольку $r(y)$ не уменьшился, а $r(x)$ — не увеличился.



Лемма о доступе

Продолжение доказательства леммы о доступе.

Случай Zig-Zig. Два вращения. Изменяются ранги только трех вершин: x, y, z .

Учетное время: $2 + r'(x) + r'(y) + r'(z) - r(x) - r(y) - r(z) = 2 + r'(y) + r'(z) - r(x) - r(y) \leq 2 + r'(x) + r'(z) - 2r(x)$, поскольку $r'(x) = r(z)$, $r'(x) \geq r'(y)$ и $r(y) \geq r(x)$.

Теперь покажем, что $2 + r'(x) + r'(z) - 2r(x) \leq 3(r'(x) - r(x))$.

Действительно, это эквивалентно тому, что $2r'(x) - r(x) - r'(z) \geq 2$.

Легко видеть, что $\log x + \log y$ при $x + y \leq 1$ имеет максимум, равный -2 при $x = y = \frac{1}{2}$. Следовательно, учитывая, что

$s(x) + s'(z) \leq s'(x)$, выполняется

$$-(2r'(x) - r(x) - r'(z)) = \log_2\left(\frac{s(x)}{s'(x)}\right) + \log_2\left(\frac{s'(z)}{s'(x)}\right) \leq -2.$$



Лемма о доступе

Окончание доказательства леммы о доступе.

Случай Zig-Zag. Учетное время:

$2 + r'(x) + r'(y) + r'(z) - r(x) - r(y) - r(z) \leq 2 + r'(y) + r'(z) - 2r(x)$,
поскольку $r'(x) = r(z)$ и $r(x) \leq r(y)$.

Теперь покажем, что $2 + r'(y) + r'(z) - 2r(x) \leq 2(r'(x) - r(x))$. Это эквивалентно тому, что $2r'(x) - r'(y) - r'(z) \geq 2$. Это доказывается аналогично предыдущему случаю, если учесть, что $s'(y) + s'(z) \leq s'(x)$.

Итак, учетное время в этом случае составляет

$$2(r'(x) - r(x)) \leq 3(r'(x) - r(x)).$$

Если теперь просуммировать полученные оценки учетного времени по всем шагам алгоритма перекося, получим оценку

$$3(r'(t) - r(x)) + 1 = 3(r(t) - r(x)) + 1$$



Теорема о балансе

Замечание

Рассмотрим последовательность из m операций над деревом, состоящем из n вершин. Если вес каждой вершины не меняется, то суммарное уменьшение потенциала не превышает

$$\Phi_0 - \Phi_m = \sum_{i=1}^n \log_2 \left(\frac{W}{w(i)} \right),$$

где $W = \sum_{i=1}^n w(i)$

Теорема о балансе

Теорема (о балансе)

Пусть T — дерево, состоящее из n вершин. Последовательность из m операций с деревом T выполняется за время $O((m + n) \log n)$.

Доказательство.

В качестве весовой функции зададим функцию $w(x) = \frac{1}{n}$. Тогда $W = 1$, учетное время каждой операции: $3 \log n + 1$. После выполнения последовательности, потенциал уменьшается на $n \log n$. Отсюда следует утверждение теоремы. $\square$

Теорема о рабочем наборе

Теорема (о рабочем наборе)

Пусть T — дерево, состоящее из n вершин. Рассмотрим последовательность из m операций. Пусть x_j — вершина, к которой осуществляется доступ. Пусть t_j — число различных элементов не равных x_j к которым осуществлялся доступ перед j -ой операцией, после последнего обращения к элементу x_j . Тогда последовательность выполнится за время $O(n \log n + m + \sum_{j=1}^m \log(t(j) + 1))$.

Доказательство.



Упражнения

- 1 Предположим мы добавили к стеку операцию `multipush`, добавляющую произвольное количество элементов. Как изменится сложность?
- 2 Рассмотрим последовательность операций, в которой стоимость операции номер i равна i , если i является степенью двойки, и 1 в противном случае. Оцените среднюю стоимость одной операции в последовательности из n операций.
- 3 Нарисуйте косое дерево, которое получается при последовательном добавлении к пустому дереву ключей 41, 38, 31, 12, 19, 8.
- 4 Построить алгоритм слияния двух деревьев.